ECHO BOT-2 PROJECT REPORT

A PREPRINT

Balar Rutvikkumar Babubhai
Master Artificial Intelligence for Industrial Applications
Methods of Digital Automation(MDIA)
OTH Amberg-Weiden
r.balar@oth-aw.de

January 18, 2026

ABSTRACT

This report details the technical implementation of the external control system for the ECHO-BOT project. The primary focus was engineering a robust data pipeline that bridges the Webots simulation environment with a user-facing web dashboard, enabling low-latency video streaming and real-time telemetry over standard web protocols. My contributions cover three core technical implementation areas: (1) a multi-threaded Flask backend acting as a protocol bridge between MQTT and WebSockets to serve as the external controller, (2) comprehensive enhancements to the Webots robot controller including optimized JPEG compression with Base64 transmission, intrigating grid-based mapping in the web controller, and autonomous battery management, and (3) a responsive, role-based frontend architecture capable of rendering high-frequency data streams for live interaction across Admin, Nurse, and Elder dashboards.

Keywords MQTT · Webots · Frontend, Backend, etc.

1 Introduction

The ECHO-BOT (Elder Care Home Operated Bot) project addresses a critical need in modern elder care facilities: the shortage of staff to provide constant, personalized attention to every resident. This project develops a mobile robot system capable of autonomous navigation, medication delivery, and real-time video monitoring. The robot operates within a simulated environment powered by Webots, where it can navigate a care home floor plan, respond to commands from caregivers, and manage its own power consumption through an Autonomous docking system. The accompanying web-based interface allows staff to monitor the robot's status and issue commands without requiring physical proximity to the robot.

1.1 My Contributions

Following the iterative development methodology, the work presented in this report was carried out in Cycle 2, with a focus on implementing the external control system and robot controller enhancements

Backend

- Flask-SocketIO server bridging MQTT (robot) to WebSocket (browsers)

Dashboards

- **Admin:** Live camera, 2D map, manual WASD control, speed slider, Force Dock, Force Stop, Robot Temperature.
- **Nurse:** Live camera, 2D map, Go to Elder (Monitor/Give Medication), Force Stop, Call Ambulance
- **Elder:** Call Nurse (categorized requests), Call Ambulance (simulated 112 call)

Robot Controller

- MQTT communication for commands and telemetry
- Battery management with auto-charging
- Navigation commands (Elder rooms, Medicine Cabinet, Dock)
- Manual control mode and Emergency Stop
- Autonomous docking on idle or low battery
- **Real-time Streaming:** Base64 camera frames and 2D map grid from Webots to web

2 System Architecture

The system follows a decoupled architecture that separates the robot control layer from the user interface layer using a message broker as an intermediary. This design provides three key benefits: (1) asynchronous buffering ensures the high-frequency robot control loop is never blocked by web client latency. (2) scalability allows the robot to publish telemetry once regardless of how many users are connected and (3) resilience means a crash in the web interface does not affect the robot's autonomous behavior

2.1 Decoupled System Architecture

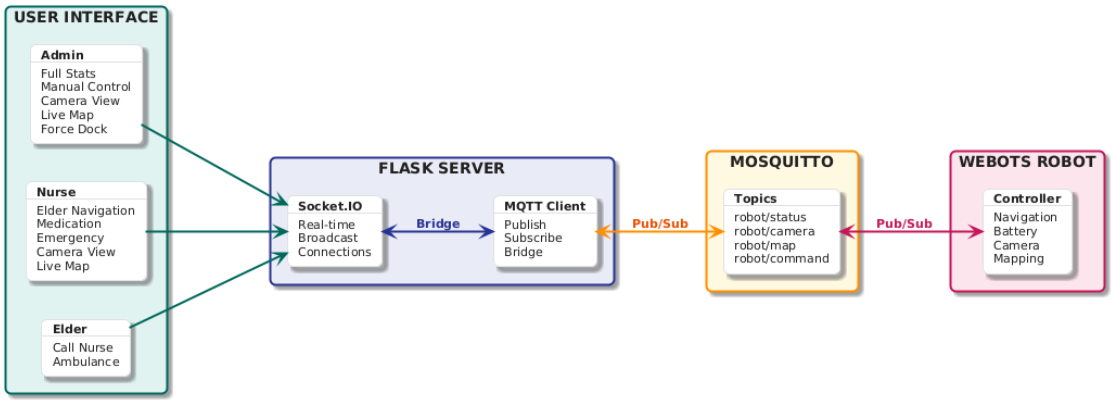


Figure 1: Decoupled System Architecture/Component Diagram

As shown in the figure 1, the architecture consists of four distinct layers. On the left, the User Interface Layer contains three role-based dashboards: the Admin Dashboard provides full statistics, manual control, live camera, map views and force dock; the Nurse Dashboard offers also live camera, map views, elder navigation, medication delivery, and emergency actions; and the Elder Dashboard presents a simplified interface for calling nurses and ambulances. These dashboards connect to the Flask Server, which acts as a protocol bridge. Internally, the server runs two modules: a Socket.IO handler that manages real-time WebSocket connections to browsers, and an MQTT client that publishes and subscribes to the broker. The Mosquitto Broker sits at the center of the communication flow, hosting four topics (robot/status, robot/camera, robot/map, robot/command) that carry all telemetry and commands. Finally, the Webots Robot Controller on the right publishes sensor data and subscribes to incoming commands, handling navigation, battery management, camera capture, and mapping logic.

2.2 MQTT Communication Protocol

The robot and server communicate through four MQTT topics:

3 Web Application Implementation

3.1 Backend Server

The backend server was implemented using Python with the Flask framework. It serves as the central communication hub between the web-based dashboards and the mobile robot. Its primary responsibilities are to receive data from the

Table 1: MQTT Topics and Data Flow

Topic	Direction	Data Content
robot/status	Robot → Server	JSON: battery, temperature, speed, task, low_battery
robot/camera	Robot → Server	Base64-encoded JPEG frames
robot/map	Robot → Server	JSON: 2D grid with robot position
robot/command	Server → Robot	Navigation and control commands

robot via the MQTT protocol and distribute this data to connected web clients using WebSockets, as well as to relay user commands from the dashboards back to the robot.

Key Technologies Used :

- **Flask-SocketIO** for real-time bidirectional WebSocket communication with web browsers.
- **paho-mqtt** library for subscribing to and publishing messages via the Mosquitto MQTT broker.
- **REST API endpoints** for handling command requests from the web dashboards.

System Operation : The backend server subscribes to three MQTT topics published by the robot: `robot/status`, `robot/camera`, and `robot/map`. Whenever a message is received on any of these topics, the server immediately forwards the data to all connected web clients using Socket.IO events. This mechanism enables real-time visualization of robot status, camera feed, and navigation map information within the dashboards.

For command transmission, the server exposes a RESTful endpoint at `/api/command`. When a user interacts with the dashboard interface, such as selecting a navigation target (e.g., "Go to Elder 1"), the browser sends a POST request to this endpoint. The server then publishes the corresponding command message to the `robot/command` MQTT topic, which is subsequently received and executed by the robot controller.

3.2 Admin Dashboard

The Admin Dashboard provides full control and comprehensive monitoring capabilities for the system. It is designed specifically for system administrators who require complete visibility into the robot's operational state and behavior in real time.

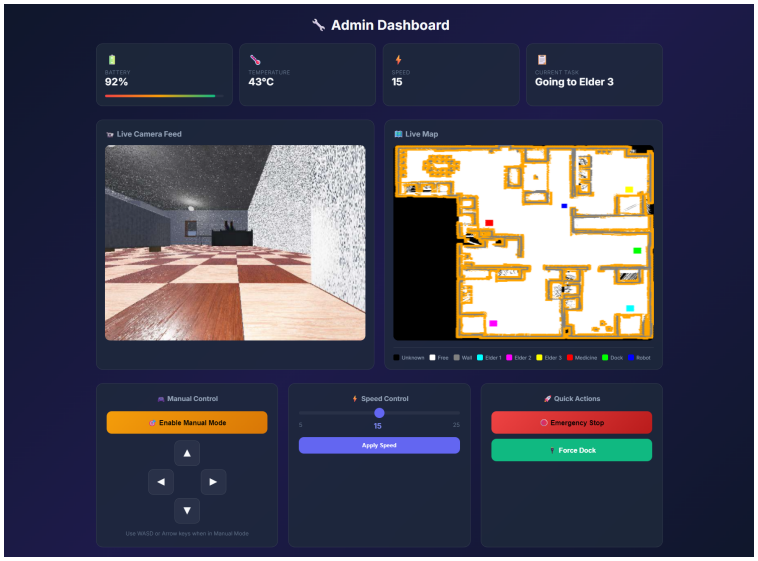


Figure 2: Admin Dashboard

The dashboard integrates multiple functional components to ensure effective supervision and direct interaction with the robot.

Features

- **Status Cards:** Display real-time system metrics including battery percentage, temperature, movement speed, and the currently active task.
- **Live Camera Feed:** Shows the video stream from the robot's onboard camera. The camera feed is transmitted as Base64-encoded JPEG images. On the client side, these images are decoded using JavaScript and rendered onto an HTML5 canvas element to provide a real-time visual stream.
- **2D Map Visualization:** Presents a real-time representation of the robot's environment using a grid-based map. Different elements such as walls, floor, docking station, and the robot itself are color-coded for clarity.
- **Manual Control:** Enables direct control of the robot using on-screen arrow buttons or keyboard inputs (W, A, S, D).
- **Speed Slider:** Allows dynamic adjustment of the robot's movement speed within a configurable range of 5 to 25 rad/s.

Map Updates : The map data is received as a two-dimensional array of integers, where each value represents a specific object in the environment (e.g. 0 = free spcae, 2 = wall, 5 = robot, 7 = docking station etc.). The client iterates through this array creating a live visualization of the robot's surroundings.

Manual Control System : The robot controller processes manual drive commands received via MQTT. When a `manual_mode` command is received, the controller pauses autonomous navigation and clears the path buffer. Movement commands (`manual_forward`, `manual_back`, `manual_left`, `manual_right`) set the motor velocities directly. The `manual_stop` command sets both motor velocities to zero. This ensures the robot responds immediately to user input from the web dashboard.

3.3 Nurse Dashboard

The Nurse Dashboard is designed specifically for healthcare staff and provides a simplified, task-oriented interface. Unlike the Admin Dashboard, it focuses on essential patient care operations while minimizing system complexity to ensure ease of use in time-critical situations.

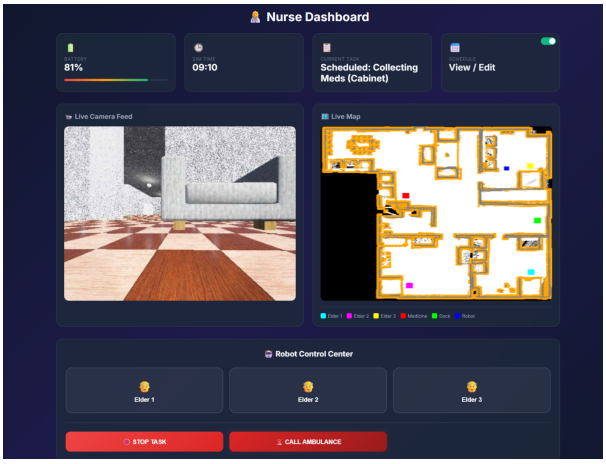


Figure 3: Nurse Dashboard main interface

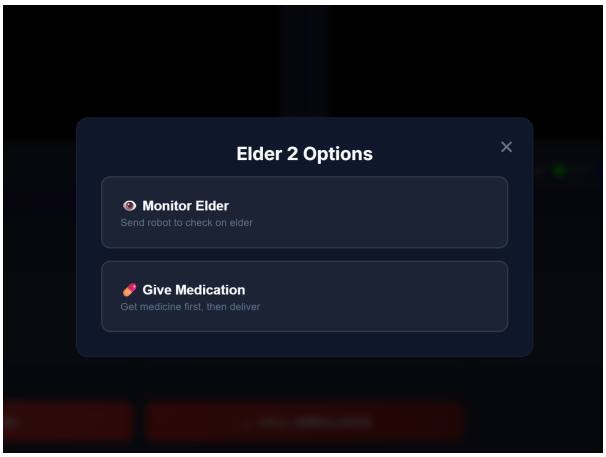


Figure 4: Nurse commands to robot

Features

- **Quick Status Display:** Shows essential system information such as the robot's battery level and the currently active task.
- **Elder Navigation Buttons:** Provides three dedicated buttons allowing the nurse to dispatch the robot directly to Elder 1, Elder 2, or Elder 3.
- **Medication Option:** Includes a selectable checkbox that instructs the robot to collect medication from the medicine cabinet before visiting the selected elder.

- **Emergency Actions:** Offers clearly visible buttons to immediately stop the current task, force the robot to return to the docking station, or initiate an emergency ambulance call.
- **Alert System:** Displays real-time notifications when an elder requests assistance, ensuring prompt staff response.

Elder Navigation Logic: When a nurse selects an elder navigation button, a dialog box is displayed with two options: *Monitor Elder* and *Give Medication*. Selecting *Monitor Elder* sends the robot directly to the chosen elder's room. Selecting *Give Medication* instructs the robot to first navigate to the medicine cabinet and subsequently proceed to the elder's room. This logic allows flexible task execution while maintaining a simple user interaction flow.

3.4 Elder Dashboard

Simplified interface for elderly users:

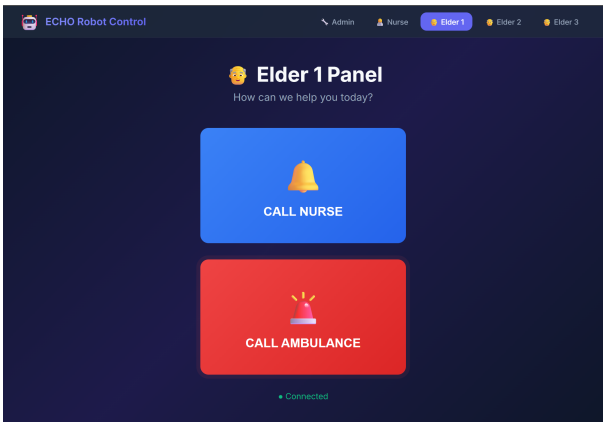


Figure 5: Elder Dashboard main interface

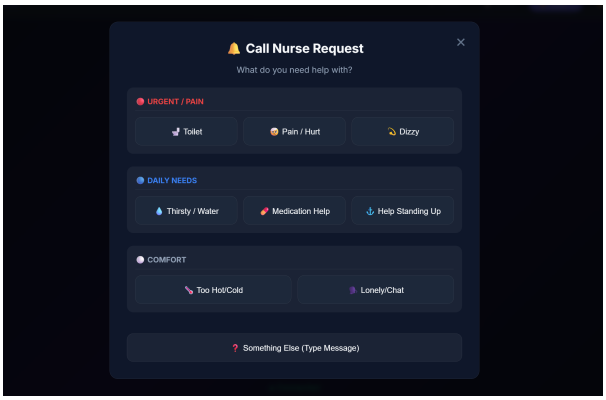


Figure 6: Elder request category menu

- **Call Nurse Button**: Opens a menu with common request categories such as *Urgent*, *Daily Needs*, and *Comfort*.
- **Call Ambulance Button**: Simulates an emergency call to **112**.

3.5 Frontend Implementation

The frontend is built using HTML, CSS, and JavaScript. The visual design uses a dark theme.

Key JavaScript Logic:

- **Socket.IO Connection**: The client connects to the Flask server and listens for events (robot_status, camera_feed, map_update)
- **Camera Rendering**: When a camera frame arrives, it is decoded from Base64 and drawn onto a canvas. This happens for every frame to create a live video effect.
- **Map Rendering**: When map data arrives, the JavaScript loops through the 2D array and draws colored squares on a canvas. Different colors represent walls (gray), floor (white), dock (green), and robot (blue).
- **Manual Control**: Event listeners detect keyboard presses (WASD) and send corresponding movement commands via the Socket.IO connection.
- **Reconnection Logic**: If the connection to the server is lost, the client automatically tries to reconnect every few seconds.

4 Robot Controller Enhancements(caretaker.py)

4.1 Battery Management System

A realistic battery simulation was implemented with three behaviors:

- **Draining**: Battery gradually decreases during robot movement at every fixed number of steps.
- **Charging**: When the robot is on the dock cell (grid value 7) and stationary, battery increases by 1% every 10 simulation steps until full.
- **Low Battery Failsafe**: If battery drops to 20% or below, the robot automatically interrupts its current task and navigates to the dock. A `low_battery` flag is sent to the web interface to trigger a warning popup.

4.2 MQTT Integration

The Webots robot controller was extended with an MQTT client using the paho-mqtt library. The client connects to a local Mosquitto broker and handles two-way communication.

Publishing: The controller publishes data on three topics:

- `robot/status` — Battery, temperature, speed, and current task (every 10 steps)
- `robot/camera` — Base64-encoded JPEG frames (every step)
- `robot/map` — 2D grid array with robot position (every 5 steps)

Receiving Commands: The controller subscribes to `robot/command` and processes incoming strings like `goto_elder_1`, `manual_mode`, or `stop`. Each command triggers a specific function (navigation, mode switch, or emergency halt).

Task Status Messages: The robot reports human-readable status strings such as "Idle", "Navigating to Elder 1", "Charging (45%)", or "Emergency Stop".

4.3 Autonomous Docking System

The robot automatically returns to the charging dock when idle to ensure it is always ready for service.

- **Idle Detection**: An idle timer increments when the robot is stationary, has no tasks, and is not charging. After 100 simulation steps of inactivity, auto-docking is triggered.
- **Navigation**: The robot uses A* pathfinding to calculate a route to the dock coordinates and navigates there autonomously.
- **Dock Detection**: The robot scans the local map grid to detect the dock cell (value 7) and adjusts its final position to ensure proper docking and charging activation.

5 Conclusion

This project developed an external control system for the ECHO-BOT elder care robot. The system consists of a Flask backend server that bridges MQTT and WebSocket protocols, three role-based web dashboards for Admin, Nurse, and Elder users, and enhancements to the robot controller including battery management, manual control, and autonomous docking, and the system enables caregivers to remotely monitor and control the robot in real-time.

6 Declaration of AI Usage

AI tools were used for debugging, code refactoring, and report writing. The core design and implementation are original work.

References

[1] Cyberbotics Ltd., "Webots User Guide," *Webots Documentation*, 2024. [Online]. Available: `https://cyberbotics.com/doc/guide/index`

[2] M. Grinberg, "Flask-SocketIO Documentation," 2024. [Online]. Available: `https://flask-socketio.readthedocs.io/`

[3] Eclipse Foundation, "Eclipse Mosquitto – An open source MQTT broker," 2024. [Online]. Available: `https://mosquitto.org/`

[4] Eclipse Foundation, "Paho MQTT Python Client," 2024. [Online]. Available: `https://eclipse.dev/paho/index.php?page=clients/python/index.php`